

EXPERIMENT 1

Student Name: Husanpreet Kaur

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70138

Section/Group: 24AIT_KRG-G2

Date: 27-07-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, self-joins, and date operations to solve real-world database problems.

Question 1: You are given with the Employee schema; the task is to find the highest score for each respective department.

Table: CREATE TABLE Employee

```
(  
    EID INT PRIMARY KEY,  
    DEPT VARCHAR(10),  
    SCORES DECIMAL(5,2)  
);
```

INSERT INTO Employee (EID, DEPT, SCORES)

VALUES

```
(1, 'D1', 1.00),  
(2, 'D1', 5.28),  
(3, 'D1', 4.00),  
(4, 'D2', 8.00),  
(5, 'D1', 2.50),  
(6, 'D2', 7.00),  
(7, 'D3', 9.00),  
(8, 'D4', 10.20);
```

SELECT * FROM Employee;

EXPERIMENT 1

	eid [PK] integer	dept character varying (10)	scores numeric (5,2)
1	1	D1	1.00
2	2	D1	5.28
3	3	D1	4.00
4	4	D2	8.00
5	5	D1	2.50
6	6	D2	7.00
7	7	D3	9.00
8	8	D4	10.20

Answer:

UPDATE Employee as E

set SCORES=(

select max(SCORES) from Employee where DEPT=E.DEPT

)

Answer:

	eid [PK] integer	dept character varying (10)	scores numeric (5,2)
1	1	D1	5.28
2	2	D1	5.28
3	3	D1	5.28
4	4	D2	8.00
5	5	D1	5.28
6	6	D2	8.00
7	7	D3	9.00
8	8	D4	10.20

Question 2: A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the **top three highest distinct salary values** within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be

EXPERIMENT 1

included in the final report. Display the department name, employee name, and salary for every qualifying employee.

Table: CREATE TABLE SalesPerson

```
(  
    seller_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50),  
    commission INT  
);
```

CREATE TABLE Company

```
(  
    com_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50)  
);
```

CREATE TABLE Orders

```
(  
    order_id INT PRIMARY KEY,  
    order_date DATE,  
    com_id INT,  
    seller_id INT,  
    amount INT,  
    FOREIGN KEY (com_id)  
        REFERENCES Company(com_id),  
    FOREIGN KEY (seller_id)  
        REFERENCES SalesPerson(seller_id)  
);
```

INSERT INTO SalesPerson

VALUES

```
(1,'John','New York',15),  
(2,'Amy','Los Angeles',13),
```

EXPERIMENT 1

```
(3,'Mark','Chicago',12),
(4,'Pam','Boston',15);
```

```
INSERT INTO Company
VALUES
(1,'RED','Boston'),
(2,'ORANGE','New York'),
(3,'YELLOW','Boston'),
(4,'GREEN','Austin');
```

```
INSERT INTO Orders
VALUES
(1,'2024-01-10',1,1,1200),
(2,'2024-01-12',2,1,800),
(3,'2024-01-15',3,2,2500),
(4,'2024-01-18',1,3,1500),
(5,'2024-01-22',4,2,700),
(6,'2024-01-25',2,3,2000),
(7,'2024-01-28',3,4,3000),
(8,'2024-01-30',4,4,200);
```

```
SELECT * FROM SalesPerson;
SELECT * FROM Company;
SELECT * FROM Orders;
```

Answer:

	seller_id [PK] integer	name character varying (50)	city character varying (50)	commission integer
1	1	John	New York	15
2	2	Amy	Los Angeles	13
3	3	Mark	Chicago	12
4	4	Pam	Boston	15

EXPERIMENT 1

	com_id [PK] integer	name character varying (50)	city character varying (50)
1	1	RED	Boston
2	2	ORANGE	New York
3	3	YELLOW	Boston
4	4	GREEN	Austin

	order_id [PK] integer	order_date date	com_id integer	seller_id integer	amount integer
1	1	2024-01-10	1	1	1200
2	2	2024-01-12	2	1	800
3	3	2024-01-15	3	2	2500
4	4	2024-01-18	1	3	1500
5	5	2024-01-22	4	2	700
6	6	2024-01-25	2	3	2000
7	7	2024-01-28	3	4	3000
8	8	2024-01-30	4	4	200

```

Answer: SELECT seller_id
FROM Orders
GROUP BY seller_id
HAVING SUM(amount) =
(
  SELECT MAX(total_sales)
  FROM
  (
    SELECT seller_id,
           SUM(amount) AS total_sales
    FROM Orders
    GROUP BY seller_id
  )
);
  
```

EXPERIMENT 1

	seller_id integer	
1		3

Question 3: A company maintains records of products sold by different salespersons. The management wants to prepare a sales performance report by identifying the salesperson(s) who generated the highest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.

Table: CREATE TABLE Department

```
(  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

```
INSERT INTO Department (id, name)  
VALUES  
(1, 'IT'),  
(2, 'Sales');
```

CREATE TABLE Employee

```
(  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary INT,  
    departmentId INT,  
    FOREIGN KEY (departmentId)  
    REFERENCES Department(id)  
);
```

```
INSERT INTO Employee (id, name, salary, departmentId)  
VALUES  
(1, 'Joe', 85000, 1),  
(2, 'Henry', 80000, 2),
```

EXPERIMENT 1

(3, 'Sam', 60000, 2),
 (4, 'Max', 90000, 1),
 (5, 'Janet', 69000, 1),
 (6, 'Randy', 85000, 1),
 (7, 'Will', 70000, 1);

SELECT * FROM Department;
 SELECT * FROM Employee;

	id [PK] integer	name character varying (50)
1	1	IT
2	2	Sales

	id [PK] integer	name character varying (50)	salary integer	departmentid integer
1	1	Joe	85000	1
2	2	Henry	80000	2
3	3	Sam	60000	2
4	4	Max	90000	1
5	5	Janet	69000	1
6	6	Randy	85000	1
7	7	Will	70000	1

Answer:

```
SELECT d.name as department,
e.name as employee,
e.salary from Employee e
join Department d
on e.departmentId=d.id
where
(
select count(distinct e2.salary)
from employee e2
```

EXPERIMENT 1

where e2.departmentId=e.departmentId
and e2.salary>e.salary
)<3
order by d.name,salary desc;

	department character varying (50) 🔒	employee character varying (50) 🔒	salary integer 🔒
1	IT	Max	90000
2	IT	Joe	85000
3	IT	Randy	85000
4	IT	Will	70000
5	Sales	Henry	80000
6	Sales	Sam	60000

LEARNING OUTCOMES:

1. Learned to use aggregate functions such as MAX(), SUM(), and COUNT() along with GROUP BY and HAVING clauses to analyze departmental performance and sales data.
2. Gained practical experience in writing correlated subqueries and nested queries to update records dynamically and retrieve department-wise highest scores and top-performing salespersons.
3. Developed the ability to retrieve meaningful information from multiple related tables using INNER JOIN operations and foreign key relationships for generating business reports.
4. Learned to solve real-world business problems by identifying the top three distinct salary values within each department, handling duplicate salary values correctly, and preparing department-wise ranking reports using SQL queries.
5. Enhanced skills in designing efficient and optimized Advanced SQL queries by combining joins, subqueries, aggregate functions, conditional filtering, and sorting techniques to generate accurate reports for employee management and sales analysis.